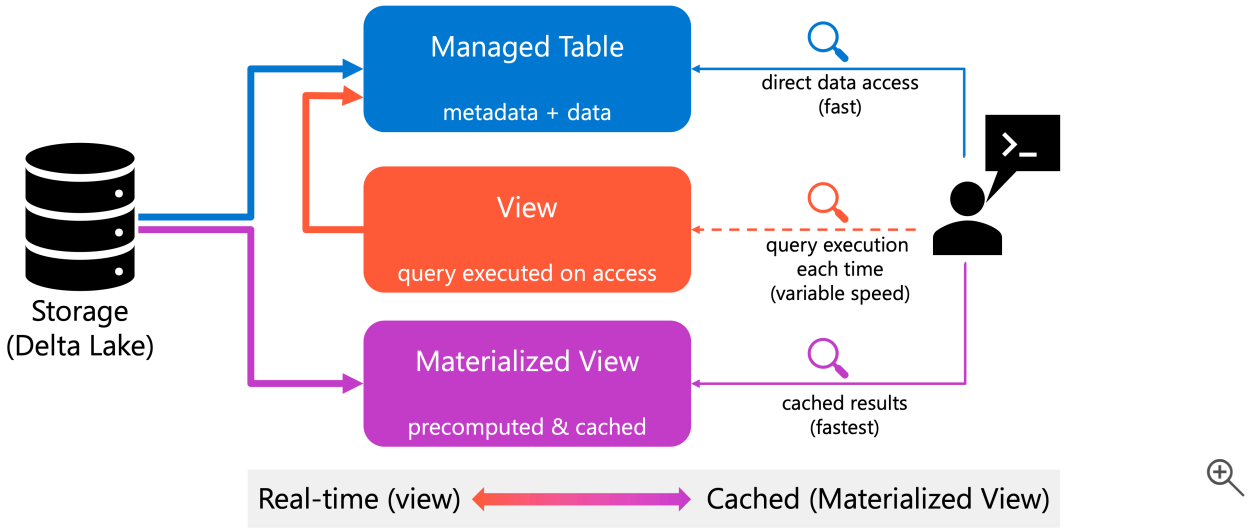
100 XP

Create tables and views

13 minutes

Tables and views form the foundation of your data organization in Unity Catalog. With tables, you persist data for long-term storage and analysis. With views, you simplify complex queries and control access to underlying data. With materialized views, you optimize query performance by precomputing and caching results. Understanding when and how to create each type helps you build efficient, maintainable data pipelines.

The following diagram illustrates how tables, views, and materialized views relate to each other and to the underlying data:



Create managed tables

Managed tables are the default table type in Unity Catalog. When you create a managed table, Azure Databricks manages both the table metadata and the underlying data files. This management includes automatic storage optimization, lifecycle management, and clean up when you drop the table.

To create a managed table, use the `CREATE TABLE` statement with column definitions. Unity Catalog uses a three-level namespace (`catalog.schema.table`), so you can either specify the full path or set your current catalog and schema context using `USE CATALOG` and `USE SCHEMA` statements. The following example creates a managed table for storing sales transactions:

SQL

```sql
-- Option 1: Use fully qualified name
CREATE TABLE production.sales.sales_transactions (
  transaction_id BIGINT,
  customer_id INT,
  product_name STRING,
  amount DECIMAL(10,2),
  transaction_date DATE
);

-- Option 2: Set current context first
USE CATALOG production;
USE SCHEMA sales;

CREATE TABLE sales_transactions (
  transaction_id BIGINT,
  customer_id INT,
  product_name STRING,
  amount DECIMAL(10,2),
  transaction_date DATE
);
```

Managed tables use the **Delta Lake** format by default, which provides ACID transactions, time travel, and schema enforcement. These features ensure data consistency and enable you to query historical versions of your data. When you drop a managed table, both the metadata and data files are deleted, preventing orphaned data in storage.

For production workloads, consider enabling features like **constraints** and **generated columns**. Constraints enforce data quality rules, while generated columns automatically compute values based on other columns. The following example shows a table with both features:

SQL

```sql
CREATE TABLE customer_orders (
  order_id BIGINT NOT NULL,
  customer_email STRING NOT NULL,
  order_total DECIMAL(10,2),
  tax_amount DECIMAL(10,2) GENERATED ALWAYS AS (order_total * 0.08),
```

```
  CONSTRAINT valid_total CHECK (order_total > 0)
);
```

Define primary keys and foreign keys

Primary keys and foreign keys help document table relationships and enable query optimizations in Unity Catalog. Available in Databricks Runtime 11.3 LTS and above (fully GA in Runtime 15.2+), these constraints are **informational only** and are **not enforced**. While they don't prevent invalid data, they provide valuable metadata for query optimization and data modeling.

To define a primary key, specify the `PRIMARY KEY` constraint during table creation. Primary key columns are implicitly `NOT NULL`. The following example creates a table with a single-column primary key:

SQL

```
CREATE TABLE customers (
  customer_id BIGINT NOT NULL,
  customer_name STRING,
  email STRING,
  CONSTRAINT customers_pk PRIMARY KEY (customer_id)
);
```

For composite primary keys spanning multiple columns, list all key columns in the constraint definition:

SQL

```
CREATE TABLE order_items (
  order_id BIGINT NOT NULL,
  item_id BIGINT NOT NULL,
  product_name STRING,
  quantity INT,
  price DECIMAL(10,2),
  CONSTRAINT order_items_pk PRIMARY KEY (order_id, item_id)
);
```

Foreign keys define relationships between tables by referencing primary keys in other tables. The following example creates a table with a foreign key constraint:

SQL

```
CREATE TABLE orders (
  order_id BIGINT NOT NULL PRIMARY KEY,
  customer_id BIGINT,
  order_date DATE,
  total_amount DECIMAL(10,2),
  CONSTRAINT orders_customers_fk FOREIGN KEY (customer_id) REFERENCES cus-
tomers(customer_id)
);
```

You can also add constraints to existing tables using `ALTER TABLE` statements:

SQL

```
ALTER TABLE orders
ADD CONSTRAINT orders_pk PRIMARY KEY (order_id);

ALTER TABLE order_items
ADD CONSTRAINT order_items_orders_fk FOREIGN KEY (order_id) REFERENCES
orders(order_id);
```

While these constraints don't enforce referential integrity at write time, they enable the query optimizer to make better decisions about join strategies and query execution plans. This can improve performance for complex queries involving multiple table joins.

Create external tables

External tables reference data stored in external storage systems while registering metadata in Unity Catalog. Unlike managed tables, dropping an external table only removes the metadata—the underlying data files remain in their original location. This separation makes external tables useful when you need to access data managed by other systems or when you want to share data across multiple workspaces.

To create an external table, specify a `LOCATION` clause pointing to your data. The location must be protected by an external location configured in Unity Catalog. The following example creates an external table over CSV files:

SQL

```
CREATE EXTERNAL TABLE archived_sales
USING CSV
LOCATION 'abfss://container@storage.dfs.core.windows.net/sales/archive'
```

```
OPTIONS (
  header 'true',
  inferSchema 'true'
);
```

External tables work best when you need to query data in its original format without moving it, or when multiple systems need access to the same data. However, you lose some benefits of managed tables, such as automatic cleanup and certain Delta Lake optimizations. For new data pipelines, prefer managed tables unless you have a specific requirement for external storage.

Create standard views

Views provide a virtual layer over your tables, defined by a query that executes each time you access the view. Standard views don't store data—they compute results on demand by running the underlying query. This design makes views ideal for simplifying complex joins, encapsulating business logic, or providing consistent interfaces to frequently accessed data.

To create a view, use the `CREATE VIEW` statement with a `SELECT` query. The following example creates a view that combines customer and order information:

SQL

```sql
CREATE VIEW customer_order_summary AS
SELECT
  c.customer_id,
  c.customer_name,
  c.region,
  COUNT(o.order_id) AS total_orders,
  SUM(o.order_total) AS total_spent
FROM customers c
INNER JOIN orders o ON c.customer_id = o.customer_id
GROUP BY c.customer_id, c.customer_name, c.region;
```

Views update automatically when underlying tables change, ensuring you always query current data. This behavior differs from materialized views, which cache results and require explicit refreshes. However, complex views can be slow if they aggregate large datasets or perform expensive joins. For frequently accessed complex queries, consider materialized views instead.

You can layer views on top of other views to create multi-level abstractions. This approach helps organize complex logic into manageable pieces. At the same time, too many view layers can make

troubleshooting difficult and impact performance. Balance abstraction with clarity and performance needs.

Create dynamic views for access control

Dynamic views extend standard views with fine-grained security controls. With dynamic views, you apply **row-level filters** and **column-level masks** based on the user querying the view. This capability lets you share data while controlling what different users can see, without duplicating tables or creating multiple views.

To create a dynamic view with column masking, use the `CASE` statement combined with the `is_account_group_member()` function. The following example shows how to redact email addresses for users not in the auditors group:

SQL

```
CREATE VIEW sales_redacted AS
SELECT
  user_id,
  CASE WHEN
    is_account_group_member('auditors') THEN email
    ELSE 'REDACTED'
  END AS email,
  country_region,
  product,
  total
FROM sales_raw;
```

For row-level security, use a `WHERE` clause with conditional logic. The following example restricts high-value transactions to managers only:

SQL

```
CREATE VIEW sales_filtered AS
SELECT
  user_id,
  country_region,
  product,
  total
FROM sales_raw
WHERE
  CASE
    WHEN is_account_group_member('managers') THEN TRUE
```

```
    ELSE total <= 1000000
  END;
```

Dynamic views require SQL warehouses, standard access mode compute, or dedicated access mode compute on Databricks Runtime 15.4 LTS or above. These security controls ensure sensitive data remains protected while still enabling broad access to analytics. Without dynamic views, you would need to create separate tables or views for each access level, increasing maintenance overhead.

Create materialized views

Materialized views precompute and cache query results, storing them as a Delta table. Unlike standard views that recalculate results on every query, materialized views return cached data, making them significantly faster for complex aggregations and frequently accessed queries. You refresh materialized views manually or on a schedule to reflect changes in underlying data.

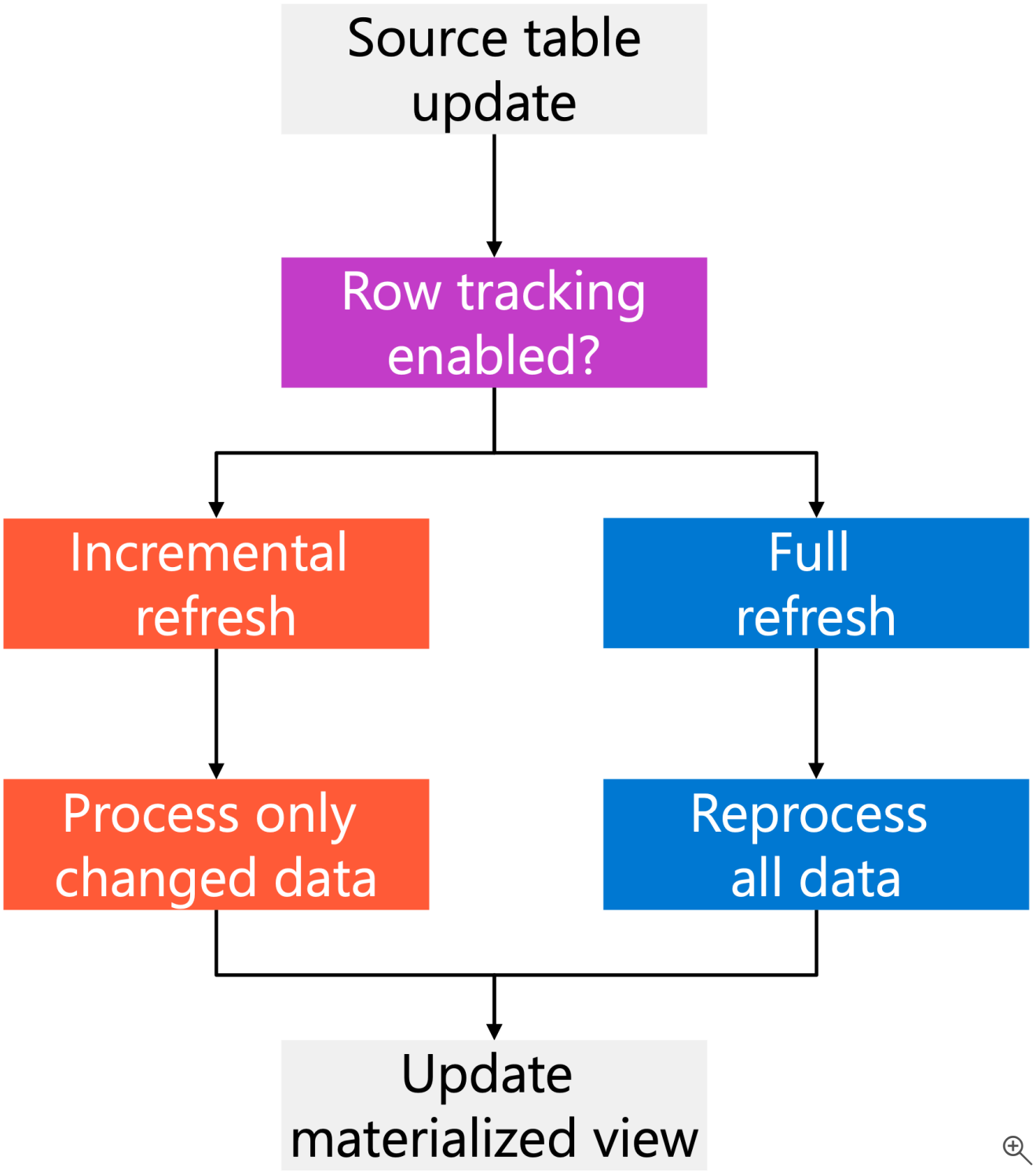
To create a materialized view, use the `CREATE MATERIALIZED VIEW` statement. The following example creates a materialized view for daily sales summaries:

SQL

```
CREATE MATERIALIZED VIEW daily_sales_summary AS
SELECT
  transaction_date,
  COUNT(*) AS transaction_count,
  SUM(amount) AS total_sales,
  AVG(amount) AS average_sale
FROM sales_transactions
GROUP BY transaction_date;
```

Materialized views support two refresh strategies: **incremental** and **full**. Incremental refresh processes only changed data, which is far more efficient than reprocessing everything. To support incremental refresh, your source tables must be Delta tables with **row tracking** enabled:

The following diagram shows how materialized views handle different refresh strategies:



To enable row tracking on your source tables:

SQL

```
ALTER TABLE sales_transactions
SET TBLPROPERTIES (delta.enableRowTracking = true);
```

> **Note**
>
> Row tracking requires Databricks Runtime 14.1 or above. Enabling row tracking on existing tables automatically assigns row IDs to all rows, which can take significant time for large tables and creates multiple new table versions. This operation upgrades the table protocol version and can't be reversed.

You can schedule automatic refreshes using the `SCHEDULE` clause when creating the materialized view, or trigger refreshes when upstream data changes. The following example schedule daily refreshes:

SQL

```
CREATE MATERIALIZED VIEW daily_sales_summary
SCHEDULE EVERY 1 DAY
AS
SELECT
  transaction_date,
  COUNT(*) AS transaction_count,
  SUM(amount) AS total_sales
FROM sales_transactions
GROUP BY transaction_date;
```

Materialized views excel at improving dashboard performance and reducing compute costs for frequently run analytics. Consider materialized views when queries take more than a few seconds to run, or when multiple users query the same aggregated data repeatedly. For data that changes constantly and requires real-time accuracy, standard views might be more appropriate despite slower performance.

Create streaming tables

Where materialized views use batch semantics to precompute results over bounded data, **streaming tables** use streaming semantics to process only new rows that arrive after the last refresh. Every streaming table is backed by a Delta table and managed by a dedicated serverless pipeline that Unity Catalog creates automatically when you run the `CREATE OR REFRESH STREAMING TABLE` statement.

Streaming tables are best suited for incremental ingestion scenarios: landing files from cloud storage with Auto Loader, consuming change events from Kafka, or building bronze and silver layers in a medallion architecture where data only ever arrives as new rows. Unlike materialized views, streaming tables don't reprocess previously seen data during a normal refresh — making them significantly more efficient for large, ever-growing datasets.

To create a streaming table, use the `CREATE OR REFRESH STREAMING TABLE` statement with the `STREAM` keyword to read from the source incrementally:

SQL

```sql
CREATE OR REFRESH STREAMING TABLE bronze_orders
  SCHEDULE EVERY 1 HOUR
  AS SELECT *
  FROM STREAM read_files(
    '/Volumes/prod_catalog/landing/raw_files/orders/',
    format => 'json'
  );
```

For production workloads where upstream data doesn't arrive on a predictable schedule, use `TRIGGER ON UPDATE` instead of a fixed schedule. This tells the streaming table to refresh automatically whenever its source tables are updated, eliminating the need to coordinate timing across pipelines:

SQL

```sql
CREATE OR REFRESH STREAMING TABLE silver_orders
  TRIGGER ON UPDATE
  AS SELECT
    order_id,
    customer_id,
    order_total
  FROM STREAM bronze_orders;
```

To manually refresh a streaming table at any time, run:

SQL

```sql
REFRESH STREAMING TABLE silver_orders;
```

Choose streaming tables over materialized views when your source only ever appends new records, when you want exactly-once incremental processing, or when the source doesn't retain full history (for example, Kafka topics). For aggregations over a complete dataset where periodic full recomputation is acceptable, materialized views remain the right choice.

Best practices for tables and views

When creating tables, start with **managed tables** unless you have a specific need for external storage. Managed tables provide better integration with Unity Catalog features, automatic optimization, and simpler lifecycle management. Enable row tracking on Delta tables that serve as sources for materialized views to support efficient incremental refreshes.

For **views**, keep the underlying queries focused and avoid creating too many layers. While layering views provides logical organization, it can make debugging difficult and hide performance issues. Document the purpose of each view and its intended audience to help other team members understand your data model.

Use **dynamic views** to implement data security policies at the view level rather than duplicating tables for different user groups. This approach centralizes security logic and reduces data redundancy. Combine dynamic views with Unity Catalog permissions to create a comprehensive data governance strategy.

When choosing between standard and materialized views, consider query performance and data freshness requirements. Use standard views for queries that run quickly or when you need real-time data. Use **materialized views** for expensive aggregations or when eventual consistency is acceptable. Monitor the refresh behavior of materialized views to ensure they update incrementally rather than fully reprocessing data.

Apply **liquid clustering** to tables and materialized views instead of traditional partitioning for better query performance and easier maintenance. Liquid clustering automatically optimizes data layout based on query patterns, while partitioning requires manual configuration and can create skewed partitions. Use the `CLUSTER BY` clause when creating tables or materialized views to enable this optimization.

Next unit: Create volumes

Previous Next